



ROI Training

Anthropic Models on Amazon Bedrock

Module 4:

Tool Use and Agentic
Workflows with Claude

Course Contents

Module 1: Foundations: Generative AI and Claude Models on AWS

Module 2: Claude API Fundamentals and Prompt Engineering

Module 3: Data Preparation and Knowledge Bases for RAG

► **Module 4: Tool Use and Agentic Workflows with Claude**

Module 5: Security, Governance, and Guardrails

Module 6: Practical Architecture and Deployment Patterns

Module 7: Claude Code: Building Bedrock Applications

Module 8: Amazon Kiro and Enterprise Code Generation



Module Objectives



Define Custom Tools

Define tools using JSON schema and integrate them with Claude



Implement Agentic Loop

Build multi-step workflows using the plan-act-observe pattern



Handle Content Blocks

Process tool_use and tool_result content blocks correctly



Design Applications

Apply tool use patterns to real-world application scenarios



Tool Definitions

What Is Tool Use?



Function Calling
for Claude

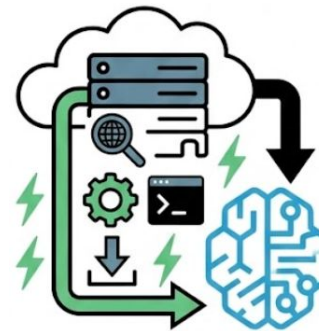
Claude decides WHEN to call a tool based on user request
Claude generates structured parameters for the tool call
Your code executes the actual operation
Claude receives results and continues reasoning

Client Tools vs. Server Tools



Client Tools

- You define and your code executes
- Custom API calls and DB queries
- Wraps your business logic
- Claude returns `stop_reason tool_use`



Server Tools

- Anthropic provides and executes
- `web_search` and `code_execution`
- `web_fetch` built into the API
- No execution code required

Tool Definition Schema

```
{
  "name": "get_customer_orders",
  "description": "Retrieves orders by customer ID",
  "input_schema": {
    "type": "object",
    "properties": {
      "customer_id": {
        "type": "string",
        "description": "Unique customer identifier"
      },
      "limit": { "type": "integer", "default": 10 }
    },
    "required": ["customer_id"]
  }
}
```

Strict Mode for Schema Conformance

Schema Enforcement

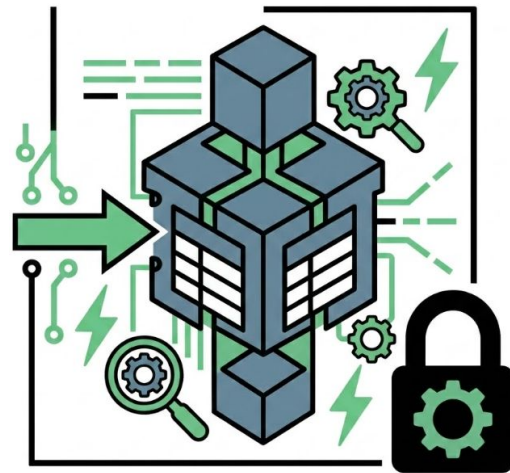
strict: true ensures exact schema matching

Prevents unexpected fields or types

Critical for automated systems

Small trade-off: slightly constrained reasoning

Production reliability is worth the cost



Model Behavior with Tools

1

Opus asks clarifying questions before calling tools

2

Opus is better for complex, high-stakes operations

3

Sonnet infers reasonable values and acts faster

4

Sonnet is better for routine operations

5

Choose model based on your use case requirements

Tool Choice Options

auto	any	tool
Claude decides whether to use tools	Claude must use at least one tool	Claude must use a specific tool
Default behavior for most apps	Response always includes tool call	Pin to one named function
Balances flexibility and control	Useful for required workflows	Great for guided flows
Ideal for conversational use	Forces structured action	Deterministic invocation

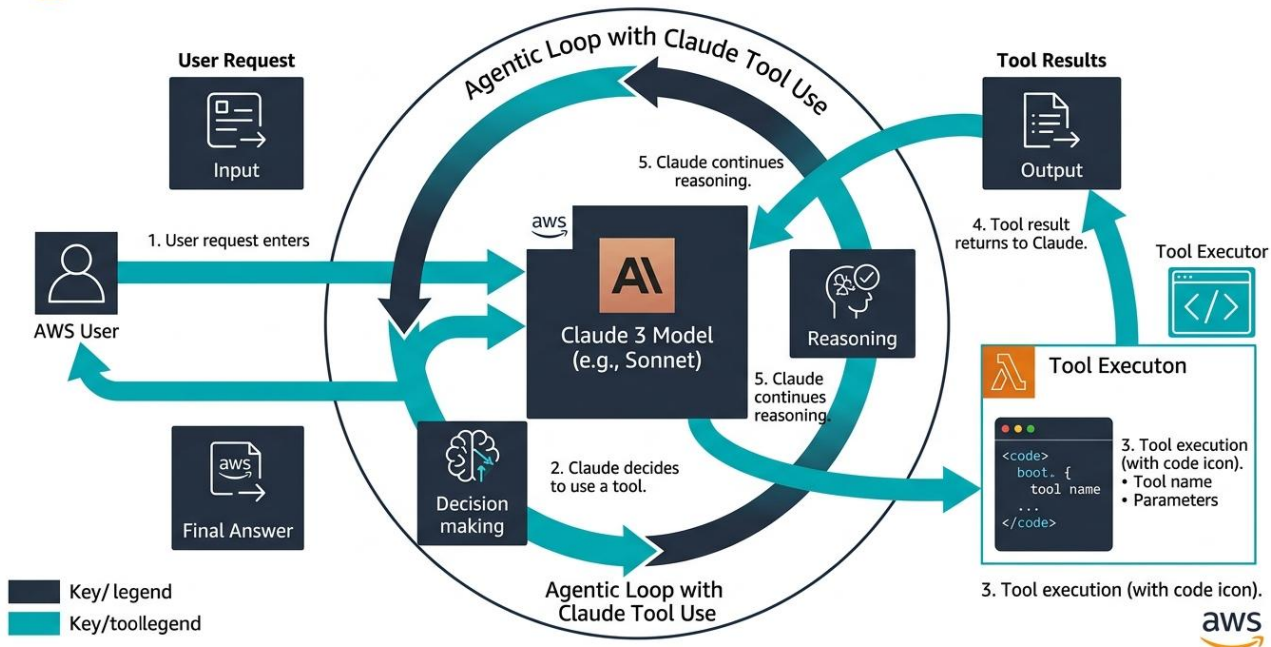


ROI Training

Agentic Loop

The Agentic Loop

aws Agentic Loop with Claude Tool Use | AWS Architecture Diagram



Agentic Loop Implementation

```
def run_agent(user_msg, tools):
    msgs = [{"role": "user", "content": user_msg}]
    while True:
        resp = client.messages.create(
            model="anthropic.claude-sonnet-4-6-v1",
            tools=tools, messages=msgs
        )
        if resp.stop_reason == "end_turn":
            return extract_text(resp)
        if resp.stop_reason == "tool_use":
            results = execute_tools(resp.content)
            msgs.append(
                {"role": "assistant", "content": resp.content})
            msgs.append(
                {"role": "user", "content": results})
```

tool_use Content Blocks

```
{
  "content": [
    { "type": "text",
      "text": "I'll look up the orders." },
    { "type": "tool_use",
      "id": "toolu_01A09q90qw90lq917835lqs",
      "name": "get_customer_orders",
      "input": {
        "customer_id": "cust_12345",
        "limit": 5
      }
    }
  ],
  "stop_reason": "tool_use"
}
```

tool_result Content Blocks

```
{
  "role": "user",
  "content": [
    {
      "type": "tool_result",
      "tool_use_id": "toolu_01A09q90qw90lq917835lqs",
      "content": [
        {
          "type": "text",
          "text": "{\\"orders\\": [{\\"id\\": \\"ord_001\\", ...}]}"
        }
      ]
    }
  ]
}
```

Demo



Multi-step Agentic Workflow

- Customer asks: "Return my last order - it arrived damaged"
- Claude calls `get_customer_orders` to find the order
- Claude calls `create_return_request` with order ID
- Claude calls `send_notification` to email confirmation
- Claude synthesizes final response

Error Handling Strategies

1

Return errors as tool_result with is_error: true

2

Include meaningful error messages Claude can parse

3

Claude can retry, try alternatives, or inform user

4

Set maximum iterations to prevent infinite loops

5

Implement timeouts for long-running tool calls



ROI Training

Patterns

Tool Design Best Practices

1

One tool, one purpose — avoid multi-function tools

2

Descriptive names: `get_order_status` not `query`

3

Clear parameter descriptions with examples

4

Return structured data Claude can easily parse

5

Include relevant context in every result

Common Integration Patterns

1

API Integration: Tools that wrap external APIs

2

Database Queries: Parameterized queries (never raw SQL)

3

Data Processing: Transform or analyze data

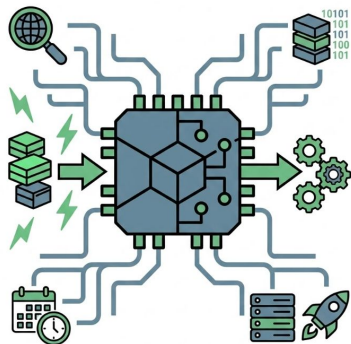
4

Notifications: Send emails, SMS, Slack messages

5

Workflow Triggers: Kick off backend processes

Amazon Bedrock Agents Overview



Managed Agentic
Workflows on
AWS

Fully managed agent orchestration on Bedrock
Action Groups define available actions via Lambda
Knowledge Base integration is built-in
Console testing with detailed trace examination

Tool Use Cost Considerations

- Tool definitions add ~346 tokens (auto/none) or ~313 tokens (any/tool)
- Each turn in the agentic loop incurs API costs
- Multi-step workflows can accumulate significant token usage
- Server tools have additional usage-based pricing
- Optimize: caching, batching, choosing appropriate models

Tool Use Security

1

Tools execute with YOUR permissions, not user's

2

Validate all inputs before execution

3

Implement rate limiting on sensitive operations

4

Log all tool executions for audit trail

5

Confirmation flows for destructive actions

What We Covered



Define Custom Tools

Define tools using JSON schema and integrate them with Claude



Implement Agentic Loop

Build multi-step workflows using the plan-act-observe pattern



Handle Content Blocks

Process tool_use and tool_result content blocks correctly



Design Applications

Apply tool use patterns to real-world application scenarios

Module 5 Is Next!

Module 1: Foundations: Generative AI and Claude Models on AWS

Module 2: Claude API Fundamentals and Prompt Engineering

Module 3: Data Preparation and Knowledge Bases for RAG

Module 4: Tool Use and Agentic Workflows with Claude

▶ **Module 5: Security, Governance, and Guardrails**

Module 6: Practical Architecture and Deployment Patterns

Module 7: Claude Code: Building Bedrock Applications

Module 8: Amazon Kiro and Enterprise Code Generation

